P1.1 — direct-Ir informed Berge search + P-graph extension audit

Aiko Seto

Created-at: 2026-09-01 20:25 CEST

Governance

`GOVERNING-STRATEGY: docs/hymeko-platform-strategy.md` · `STRATEGY-ID: HYMEKO-PLATFORM-2026-09-01-v` · `ROADMAP-TASK: P1.1` · `APPROVED-GRAPH-EDIT: direct-ir-informed-berge-search`. Strategy ID not bumped. **plan.pdf waiver**: no TeX engine on this host (tools.yaml `on_absent_tool: document_waiver_in_report`); `plan.tex/.tikz/.mmd` authored compilable, pdf not built.

ETA

≈4–5 h: promotion gate 15 min (done); reading + audit 45 min (done); plan bundle 30 min; baselines 30 min; A* + heuristics + policy implementation 90 min; oracles/fault-injection 60 min; benchmark 40 min; D*/LPA* decision + prototype 30 min; pgraph audit doc 20 min; V&V + reports + push 45 min. Basis: the search reuses the existing incidence primitives; the risk is entirely correctness (oracle parity) and the warm-alloc-free contract, both machine-gated.

Scope / goal

An *informed* Berge-search layer over the canonical `Ir` + exact sparse `IncidenceIndex`: Dijkstra/uniform-cost baseline, A* shortest Berge path, metric and hypergraph-discrete heuristics, a deterministic caller-owned workspace, warm-query allocation-free, and **no** materialised Levi/star/clique or second graph model. Plus a **read-only** architectural + performance audit of making `hymeko_pgraph` a true domain *extension* of `hymeko_graph` (general navigation/search owned by graph; classification/axioms/MSG/SSG/ABB/cost owned by pgraph). The D* Lite/LPA* line is **designed and benchmark-prototyped only**, not wired to production IR mutation (no canonical transaction/snapshot layer yet).

P1 promotion gate (§0, executed)

Verified 4 refs by `ls-remote`; ancestry `2ebb833`≺`5fe2c88`, `cbaadd4`≺`c35b9f2`; re-ran the P1 targeted gates (fmt/clippy/KAT/pgip) green; fast-forward-only promotion `core main` → `5fe2c88`, `graph main` → `c35b9f2`. New branch `feat/direct-ir-informed-berge-search` off the promoted graph main. Core production source untouched.

Semantic contract (see semantic-contract.md)

State space: implicit incidence `node` → `(arc,slot)` → `node`; forbidden: explicit Levi/star/ clique, dropping sign/weight/slot/duplicate, dense-id reinterpretation. Cost: typed, finite, non-negative (`NaN`/$\pm\infty$/negative = typed error), exit-independent-separable primary model ($c = w_{arc} + w_{entry}$) enabling arc-activate-once (O(incidences)); metric/general path relaxes per member. Policy: undirected / sign-compatible / directed producer→consumer / consumer→producer / domain — filter only, chosen once (no runtime mode boolean, no `Box<dyn>` in the hot loop). Order: `f64::total_cmp` then `(discrete_tiebreak, DeclId, HyperArcId, slot)`. Reopen: consistent- heuristic fast path; an inconsistent heuristic in optimal mode is a typed rejection (fail-closed), not a silent sub-optimal path.

Affected files (new; additive)

`hymeko_graph/src/direct_ir/informed.rs`	NEW: workspace, cost/heuristic/policy traits,
`hymeko_graph/src/direct_ir/mod.rs`	+`pub mod informed;` + re-exports
`hymeko_graph/tests/informed_search_vv.rs`	NEW: oracles, equivalences, boundary, fault-i
`hymeko_graph/tests/informed_search_bench.rs`	NEW: latency / expanded-state / heuristic-qu
`hymeko_graph/tests/informed_search_alloc.rs`	NEW: warm-query zero-alloc (isolated single-t
`docs/pgraph-as-graph-extension.md`	NEW: read-only audit narrative
`reports/2026-09-01-direct-ir-informed-berge-search.md`	NEW: P1.1 verification report

CORE.YAML items touched: **none** (`hymeko_graph` is core-listed in graph CORE.YAML; this is an **additive** new module under the `direct-ir-informed-berge-search` token — no existing public API changed, no dependency added). `hymeko_pgraph` not modified (read-only audit). Core repo untouched.

Test / coverage / benchmark matrix

Oracles: unit-cost = BFS `berge_shortest_path`; weighted = a test-only Dijkstra on the same implicit state space; tiny graph = exhaustive simple-Berge-path enumeration; metric = closed-form. Equivalences: `astar(Zero)`=Dijkstra, unit=BFS length, admissible=same cost. Boundary: 0/1/self/singleton/empty/disconnected, signs, f32/f64 edges, `NaN`/∞/negative rejection, arity 0/1/63/64/65 + large, declaration-order permutation, disjoint-union metamorphic. Fault-injection (no mutant committed, git-clean restore): heuristic overestimate, wrong predecessor slot, skipped reopen, broken tie-break, sign-policy inversion, duplicate-incidence dedup, stale workspace generation, dropped queue key-update — each caught by a named test. Benchmark: workload-matrix.md; per-cell expanded-state, queue ops, allocations, retained bytes, peak RSS, heuristic precompute (separate), time-to-first, total, path cost/hash. Coverage: new production lines by the V&V tests; touched-line target 100% or a line-justified fail-closed exception; before/after test population reconciled by name.

Performance budget / gates (§12)

Correctness first (optimal cost identical on every oracle cell). A* production-retention requires: warm query allocation-free; no materialised adjacency; on a metric-friendly or discrete-friendly

workload $\geq 2\times$ fewer expanded states *or* $\geq 20\%$ wall improvement vs Dijkstra; on an uninformative heuristic $\leq 10\%$ overhead vs the Dijkstra engine (or a separate zero-heuristic fast path); memory $O(\text{nodes} + \text{arcs})$; **no** $> 3\%$ regression in any untouched direct_ir op. The goal is that the *benchmark selects* where a heuristic pays — not that every heuristic always wins.

D* Lite / LPA* decision (§8)

Decision matrix: static+single-query → A*; same goals, changing cost/topology → LPA* candidate; moving start → D* Lite candidate; AND/OR producibility → separate domain model (not simple Berge D*). No production incremental state (no snapshot/transaction layer): a **benchmark-only** prototype may use two immutable before/after `Ir` snapshots + an explicit `ChangedIncidence` list (not called a canonical delta), query-workspace state only (`g, rhs, queue, version`), no second adjacency. Measure fresh-A* vs replan for 1/10/100/1000 updates + start movement; repaired = fresh Dijkstra/A*; report break-even query count. Honest **DEFERRED** if no clear win.

Rollback / HARD STOP

Additive on one feature branch; not promoted. Drop on: oracle disagreement, warm-query allocation, a forbidden artefact (Levi/star/clique on the standard path, second engine, `Box<dyn>` hot dispatch, `partial_cmp` on cost, an `IRDelta`/snapshot framework, production D*), a $> 3\%$ untouched-op regression, or any pgraph/core/Nagare/Hotaru production modification. **HARD STOP** at P1.1 end: no production PGRAPH migration, no P2 operator-precompute.

Indexed frontier continuation, 2026-10-12 01:49 +02:00

User approved all candidates in the Core algorithm/data-structure catalogue. Core first: shared Index/imports, incremental symbol counts and linear parent classification pass 193 library and 181 IR tests, plus Clippy. This continuation supersedes the original implementation stop, not the Git publication gate. The checkout advanced externally to f7ed8c2 (orthtree merge plus the earlier arena-test adaptation); freshly fetched feature upstream matches. Reuse this checkout and preserve unrelated work. No modifications in the target informed search file appeared during that integration.

Owner: existing `hymeko_graph/src/direct_ir/informed.rs`. Discovery found lazy BinaryHeap queues in top-k modules but no indexed/decrease-key heap; those path-state algorithms are not substitutes. The current search already keys g, predecessor and settled state by DeclId, making one frontier entry per ID valid. Retain Entry/Key ordering and the one production A-star/Dijkstra entry point.

Use a Vec of Entry plus a full-width inverse position Vec. Membership checks the position against live heap length and the entry's DeclId. Clear only resets the trivial-entry Vec length; stale inverse entries are harmless. Ensure both arrays for the declaration domain before search. Insert/update/pop are O(log N), clear/membership O(1), live frontier at most N. Preparation and memory are O(N), even when the actual frontier is small: this is an explicit tradeoff. No further heap growth inside search. Existing generation-stamped state still has a wrap clear; this change does not remove that separate cost.

Push statistics continue counting successful relaxations plus the start entry, including priority updates. Pops/stale-pops count actual extractions and decrease. Preserve path costs, policies, tie

choices and errors. No signature or dependency change. Touched: this bundle, CORE.YAML, SYMBOLS.md if generated output changes, and `hymeko_graph/src/direct_ir/informed.rs`. Capture the source before edits. Private queue tests use the minimum scan over one optional entry per ID as a small independent oracle, not another search engine. Cover both key directions, root/last removal, clear, regrowth and reuse. Run existing public path/oracle and allocation checks, Graph correctness/strict Clippy, then Web/WASM consumers. Use one local Core identity; temporary lockfiles restored only after hash checks. No timing matrix. Resource cap 16 GiB. ETA 15 minutes (implementation 5, tests 5, shared PDF/report 5). Roll back only these named hunks. TeX is available here; build plan.pdf before edits. The original PDF-tool waiver is historical.

Verification closure, 2026-10-12 02:05 +02:00

Implemented one private indexed frontier. Three new tests cover 6,144 mixed queue operations against a minimum-scan oracle, equal-key payload replacement, and an integrated improved-route query with four updates, three pops and no stale pop. Full workspace lib/tests: 644 passed, 30 ignored. Strict Clippy, formatting and symbol index checks pass. Web/WASM's 42 native tests and wasm32 check pass with one local Core/Graph identity; original lockfiles restored.

Duplication gate remains a pre-existing failure: the checked-out HEAD f7ed8c2 and candidate both measure 577 clones / 13,157 lines, against stored 547 / 12,629. The existing detector was used with the HEAD version of the single changed source substituted only in memory. Zero added duplication; baseline unchanged. Evidence and shared PDF: `reports/2026-10-12-indexed-frontier/` and `reports/2026-10-12-indexed-frontier.pdf`. No latency gain is claimed. The original source SHA-256 starts `0e8d885d2e64847f`. No public API or dependency change here; observable pops now reflect fewer stale extractions. No commit or publication by this task; preserve the externally integrated branch.

Sparse visitation continuation, 2026-10-12 05:20 +02:00

User authorized all catalogue algorithms and requests evening completion. Core correctness and Clippy gates passed first. Replace generation markers in TraversalWorkspace and InformedSearchWorkspace with one private typed sparse set in the existing workspace module. Dense IDs double as traversal discovery order; full-width inverse positions are validated against live dense entries. Logical clear is worst-case O(1), with no generation rollover. Ensure reserves the whole ID domain; this increases memory versus 32-bit stamps and is stated explicitly. Informed g membership guards settled booleans; first discovery resets settled. Keep heap, tie order, paths, errors and public signatures unchanged. Search found no reusable visited-set owner; IndexedFrontier has heap positions, not visitation. Tests cover stale positions, adversarial reuse, growth, both ID domains, settled state and existing zero-allocation/path oracles. No latency claim. ETA 20 minutes, 16 GiB cap. Preserve pre-existing TeX artifacts. Roll back only these named hunks. Report joins Core's algorithmic-completion PDF; no commit/push authorization.

Shared search/evidence runners: 2026-10-14 21:06 +02:00

User approved the next complete family: A*/Dijkstra test execution, bounded path/cycle evidence, library heuristics and a real declaration-order permutation. ETA about 70 minutes: 20 library/support design and migration, 25 consumer wiring, 15 focused tests/strict gates, 10 PDF/archive. No commit, push, merge or benchmark run.

Ownership: Graph informed.rs owns borrowed HopHeuristic and MaxHeuristic. Hop estimates require exact undirected DeclId-indexed distances for the same IR/goal and a valid lower bound on step cost; numeric construction guards reject negative/nonfinite/overflowing estimates. The maximum composes child guarantees by conjunction and retains the default zero tie-break. No new traversal engine.

Existing informed_common gains SearchCase plus static SearchRunner strategies for the two existing public engines. Cases borrow the original workspace: no hidden fresh workspace, no change to reuse/statistics tests or timing boundaries. The evidence target gains one operation/sampler verification protocol with weighted-search and typed path/cycle operations. Calibrated measurement remains owned by performance_evidence/statistics.rs; a deterministic sampler tests the protocol without taking timings. Keep result hashes outside timing, cheap sentinels inside, result destruction and final work-counter comparison unchanged. Weighted scratch is retired before formatting, matching the previous helper scope. Enumeration emits its prepared phase before subject dispatch, including invalid-subject diagnostics. Typed cell/result context consolidates output; preserve TSV keys/order/units and hash domain bytes. Existing measurement entry tests remain ignored. Audit correction: a pre-existing unignored parsing test called real nine-sample timing factories during initial checks. Move it behind explicit ignore/opt-in, preserve it, and add an untimed parse/resolve/lower/compile/index/file/query parity test. Do not use those initial timings for performance claims.

Permutation: reuse direct_ir_common::compile on two source orderings, changing both declarations and edge ordering. Let canonical compilation remap references. Assert actual ID changes, normalized edge/reference parity, named-endpoint Dijkstra/A* costs, unreachable cases and a control that fails if IDs are reused without remapping. Keep independent oracle algorithms independent.

Files: src/direct_ir/informed.rs and mod.rs; tests/informed_common/mod.rs and new search.rs; tests/informed_search_vv.rs, map_routing_nav.rs, informed_berge_evidence.rs and execution/reporting support; informed_common/parsing.rs (dependent row consumer), informed_search_alloc.rs (same zero-allocation invariant through the new strategies); shared MapPositions replaces the two identical fixture providers; CORE.YAML, SYMBOLS.md, README.md, this four-format plan and the task PDF. Core's existing unpublished APIs are consumed through the explicit local Cargo patch. Core source is not changed. Capture and restore the exact Graph lockfile.

Acceptance: all equivalent heuristic copies and evidence execution branches migrated; old counted/hash fixtures frozen before replacement; typed rejection, flag propagation, boundary tables, consistency/admissibility and parser-backed metamorphic assertions pass. Mock sampling validates sample/hash/stat drift rejection and fixed output rows. Strict Clippy, public symbols and unchanged clone gate. No benchmark/profiling or speedup claim. Compilation budget 16 GiB; small deterministic fixtures. Scratch lifetimes and caller budgets remain intact. Risk: wrong ID domain, false consistency flags, extra work inside measured callbacks, changed hash domains or TSV order, accidentally testing the engine against itself. Rollback is this task's captured diff only. Fresh fetch succeeded on both remotes for Graph and Core; HEADs 921d1d9 / fceee42, existing task branches/checkouts reused.

PageRank and active graph workloads

Created-at: 2026-10-17 21:21 +02:00. ETA 50–80 minutes: 20–30 min implementation and independent numerical checks, 10–15 min correctness/lints, 5–10 min bounded measurements, 15–25 min provenance, PDF and archive. Explicit user request: implement PageRank in Graph and characterize work performed on a resident graph, not only construction. User selected an undirected vertex–hyperedge–vertex walk as the default. The same iteration supports explicitly selected sign/port routes.

Discovery and ownership. Graph direct_ir owns canonical-IR traversal. BFS, Berge shortest paths, Dijkstra/A*, components and reusable scratch already exist. No PageRank implementation was found in Graph, Hotaru or Tensor sources. The existing signed f32 uniform-arity operator is not a probability operator and is not substituted silently. Add one PageRank owner under direct_ir; no Core source, dependency, RL, alternate graph model or IR builder is introduced. GraphGenerator/IrBuilder and Core measurement/allocator utilities are reused by clients. Existing performance_evidence statistics, telemetry and process runner remain measurement owners. The current Graph checkout (921d1d9, feat/newest-core) and Core checkout (fceee42) are reused, preserving prior dirty work. Fresh fetch succeeded on origin/github; divergence from main is Graph 1/4 and Core 2/6.

Exact semantics. Let a(v,e) be a nonnegative source incidence mass and b(v,e) a nonnegative destination mass, counting duplicate slots. Select an active incident arc proportionally to a, then a member proportionally to b. An arc with zero destination mass cannot be selected; a node without active outgoing mass is dangling. Default: unit masses, every sign/port allowed, including a return to the same vertex. This is the Zhou et al. (NIPS 2006, section 4) incidence walk, with PageRank teleportation. Directed pairwise arcs with one source and destination recover ordinary directed PageRank. Do not claim the undirected return-permitting walk equals no-self-loop pairwise PageRank at the same damping. Optional routes: minus-to-plus and In-to-Out, independently selectable; no inference of port direction from sign. Weights are explicitly unit or one numeric vector component; absent vectors default to one, present missing/nonnumeric/negative or nonfinite components fail. Personalization is normalized, finite, nonnegative and nonzero. Dangling mass follows that same personalization.

API and complexity. PageRank::build borrows immutable Ir and validates node targets, then prepares only degree/arc normalizations and optional numeric weights. PageRank::run accepts typed iteration configuration, reusable workspace and optional personalization. No clique/adjacency expansion. Each iteration is O(V+E+I); scratch O(V), preparation O(V+E), plus O(I) only for component weights. One iteration loop owns all routing variants. Compensated sums protect long reductions. Report iterations, L1 step change, convergence and the contraction error bound alpha/(1-alpha) times the last change. Iteration cap returns a clearly nonconverged result. Finite alpha in [0,1), positive tolerance and nonzero cap. Empty graphs and alpha zero have explicit zero-iteration results.

Files. CORE.YAML approval entry; direct_ir/pagerank.rs and a focused private helper module if needed; direct_ir/mod.rs and lib.rs exports; tests/pagerank.rs, tests/pagerank_support/mod.rs, tests/pagerank_alloc.rs; existing performance_evidence.rs with a ranking client under performance_evidence/; existing campaign runner only if required; README and generated SYMBOLS; this four-format

plan and the task PDF/raw evidence. User approval explicitly covers hymeko_graph addition. No runtime dependency changes.

Correctness. Closed-form chains, cycles, weighted hyperedges and isolates; independently materialized small transition matrices and Gaussian linear-system solve; deterministic generated/permuted/duplicate cases; mass and nonnegativity; sign/port independence; scale invariance; bad inputs; nonconvergence; workspace reuse and deterministic zero warm allocations. Fixtures invoke Core's single IrBuilder/IrConstruction owner. Existing topology remains unchanged.

Measurements. Extend the existing opt-in evidence client. Deterministic connected synthetic workloads: 65,536 vertices, approximately one million incidences, arities 2/32/1000; add the previous disjoint 1000-by-1000 shape as a control, explicitly separated. Measure PageRank preparation and warm solve and existing reachability; construction and index preparation excluded from operation time but included in process peak memory. Use two warmups, nine raw samples per cell, five fresh-process repetitions, one CPU thread, exact fixtures/seeds and source/binary hashes. Run a full-scale smoke before the matrix. Existing sampler and process telemetry, no new timer/OS implementation. Per-operation budget 15 s, per-process 180 s, peak 16 GiB. Inspect quiet-host CPU before launch; sequential measurement processes. No external system or world-fastest claim without matched semantics/workload. No profiler is authorized by this step and no profile-based micro-optimization is performed. PDF plus plotted and raw numerical evidence.

Risks and rollback. Source slots can duplicate a target; node IDs differ from declaration IDs; singleton/self-return semantics must not drift; inactive arcs must not lose mass; high alpha may not converge by the cap; sums can overflow. Reject invalid inputs and preserve finite bounded results. Local Core API remains unpublished, so record a temporary CLI patch and restore Cargo.lock exactly. No commit/push is requested; no merge or branch switch. Preserve initial files and a task-only patch for rollback. Archive with the existing ReportArchive tool and verify bytes before reporting completion.

Matched native comparison and default-path audit

Created-at: 2026-10-18 02:06 +02:00. ETA 60–90 minutes. User requests fresh numbers and the better implementation as the default. Preserve semantic defaults: undirected/unit PageRank remains selected unless the caller requests directed routing. No precision relaxation, legacy IR builder, new production backend, profiler, commit, push, RL edits, or other-agent work is included.

Construction. Build the current Core benchmark using the sole SyntheticIrBuilder/IrConstruction path and compact 32-byte references. Compare against the unchanged archived hypergraphz native adapter at pinned commit e45b7b80e3ce302937a56760d9fe76016ef08e26 on 1,000 disjoint hyperedges of 1,000 vertices each. Verify every member and normalized checksum before timing. Five fresh processes per backend, build and build/drop, CPU affinity, raw Criterion samples, peak memory and source/binary hashes. Historical figures are identified as historical and are not substituted for fresh same-host results.

Active operations. Extend the existing Graph evidence client and process runner with a benchmark-only native ABI calling unchanged upstream computePageRank and getConnectedComponents. Both use identical canonical fixture membership exported outside timing. Uniform personalization, unit weights, damping 0.85, global L1 error bound 1e-10, cap 200; upstream iterate-difference threshold is converted by (1-alpha)/alpha. Distinct members per edge avoid the upstream set-incidence versus HyMeKo multiplicity difference. Compare full rank vectors and normalized whole-component partitions outside timing. Compare one-shot PageRank including preparation/workspace/result cleanup on both sides; show HyMeKo reusable-workspace solve separately with its boundary explicit. Reverse indexes are built before component queries on both sides. Upstream BFS follows ordered pairs and is not equated with HyMeKo hyperedge reachability.

Measurements and gates. Use existing sampler (two warmups, nine samples), five fresh processes per cell, one CPU, 16 GiB observed process guard, 180-second process timeout, and source drift checks. Start with a full-scale smoke. Fixtures: 65,536 vertices, one million memberships, arities 2/32/1000. No uniform regular-disjoint PageRank speed claim: its uniform start is already stationary. Small native parity tests cover isolates, singleton/empty edges, asymmetric degrees, and disconnected components. Existing directed correctness stays active; directed external comparison is inapplicable because upstream PageRank is unsigned. Run scoped correctness, Clippy, formatting, Python controller tests/types/lint, symbol and duplication gates. No production optimization without evidence.

Delivery. Audit normal callers and default configuration, preserving one production construction and PageRank implementation. Record fresh Git fetches, dirty source identity, temporary local Core Cargo patch and restored lockfile. Produce PDF, raw numerical/source/binary evidence and byte-verified shared archive. Reuse the current checkouts and shared reporting tools. A slower result is reported as such; selecting different mathematics is not an optimization.

Completed comparison, 2026-10-18 02:31 +02:00. Completed 75 Graph measurement processes (675 samples), five smoke cells, and 15 construction processes (600 samples), with full result/source checks. Current construction 80.03 ms versus archived builder 79.83 ms; build/drop 85.09 versus 93.14 ms. Native hypergraphz construction 61.07 ms. Components favor HyMeKo

(2.17/6.06/89.19 times); PageRank favors the pinned upstream (7.63/4.57/3.13 times), at identical iteration counts and precision. The small singleton-edge component case exposes an upstream short-window indexing limitation; the adapter explicitly rejects that operation, without modifying the upstream algorithm. All timed cases are within the common domain. Production sources are unchanged, one canonical owner remains the local default. Graph's published Core pin is still old; this run records a temporary local patch, not a completed publication. No profiler, production optimization, commit or push.

Shared virtual incidence preparation and PageRank integration

Created-at: 2026-10-18 17:29 +02:00. ETA 45–75 minutes: shared projection and operator integration, directed/numerical/storage regression gates, PDF and archive. Authority: the user explicitly requests applying the shared virtual sparse operator approach to PageRank, building on the existing Core owner. Earlier compact-reference approval covers correcting the demonstrated retained-capacity error. No benchmark, profiler, commit, push, RL or other consumer migration is included. This is an ownership/integration change with structural correctness checks; no speed factor will be claimed without a later authorized comparison.

Discovery. IncidenceIndexData owns reverse CSR coordinates; PreparedIr/IncidenceView bind it to canonical Ir. IncidenceOperatorPlan already prepares signed transpose coefficients. PageRank currently owns repeated target resolution and per-incidence role dispatch. Extend this family; pairwise Csr and uniform-arity f32 SignedHyperIncidence have different contracts. The new private incidence/projection.rs submodule keeps the existing large owner readable; its public types are re-exported by ir::incidence. No new crate or dependency.

Shared representation. Add IncidenceProjection with immutable, reference-counted arc offsets and native-size numeric target indices, paired with its Ir. Explicit domains are canonical declarations or canonical nodes. Native usize indices preserve existing size limits; no unchecked u32 narrowing. Validate the selected target domain once, preserving order and duplicate slots. The existing reverse index lazily caches each projection domain, sharing only structural data across topology-preserving model updates. Signed coefficients stay attached to the operator/Ir, never in the shared structural cache. A direct projection build avoids forcing a reverse index on an algorithm that needs none.

Shared execution. IncidenceRow owns gather/scatter primitives with a small accumulator trait: ordinary f64 for the existing signed operator, the existing compensated sum for PageRank. Existing IncidenceOperatorPlan gains lazy forward coefficients derived from its checked transpose coefficients and uses these shared rows for Bx. Its transpose implementation stays intact. PageRank uses the same row operations, fusing each arc's gather and scatter to avoid an extra arc-sized temporary. No B-transpose-B or clique is materialized.

PageRank. The ordinary build method creates the Core node projection; a constructor from an existing projection permits reuse by multiple operators. Remove per-iteration DeclId decoding and sign/port matching. Prepare unit, directed-unit role bits, or normalized component rates once. Dispatch outside the incidence loop. Preserve self-return, multiplicity, independent sign/port semantics, selected vector component, dangling handling, personalization, compensated arithmetic and convergence bounds. Preserve original fold and multiply order; no precision relaxation or retained old/new kernel switch.

Storage correction. store_references currently reuses the 96-byte ReferenceDraft allocation for 32-byte records: 1000 records retain capacity 3000. Compact the completed vector at the canonical owner boundary; allocation may be resized or moved once. Test the retained IR vector directly (not a clone), payload presence, wire/hash semantics and empty input. Existing IR record identity is unchanged. Do not add a per-consumer shrink workaround.

Files and tests. Core: CORE.YAML, ir/incidence.rs, its projection submodule/tests, ir/reference.rs and reference/tests.rs, existing prepared-IR and precompute correctness tests, README/SYMBOLS and this plan. Graph: CORE.YAML, direct_ir/pagerank.rs and module documentation, existing pagerank.rs and pagerank_alloc.rs tests, README/SYMBOLS and this plan. Keep frozen earlier measurement artifacts. Add Core domain/cache/adjoint/duplicate/empty/invalid projection tests and shared prepared-forward parity. Graph independent dense oracles cover all route/weight modes, isolated/interleaved IDs, extreme weights, nonconvergence and guards; add shared-projection parity and actual default-path allocation checks. Warm applies and rank solves remain allocation-free.

Verification and resources. Inspect exact test targets; run only non-measurement correctness tests, compilation, Clippy, fmt, symbols and duplicate gates. Two Cargo jobs, bounded deterministic fixtures and the 16 GiB process cap. Graph uses a recorded temporary local Core patch and restores Cargo.lock bytes; the older published dependency pin is not misrepresented as updated. Extra projection storage is O(I+E); cache population and failures are explicit. No retained payload copies, global registry, unsafe indexing, dependency cycle or new IR builder. Wrong-domain or stale pairing must not silently succeed.

Git and rollback. Fresh origin/github fetch succeeded in both repos; Core fceee42 is 2/6 and Graph 921d1d9 is 1/4 versus main. Each has one worktree. Preserve all earlier dirty work. Before-files and task-only patches live under the existing comparison report family's shared-incidence phase. Report PDF, source/binary/test receipts and byte-verified ReportArchive copy close the task.

Measured shared-incidence PageRank comparison

Created-at: 2026-10-18 18:31 +02:00. ETA 15–25 minutes: one existing five-cell smoke, then five fresh processes per applicable comparison cell, validated summaries, PDF and archive. Authority: the user requests continuing the competitive performance work against hypergraphz after accepting shared IR.

Scope. Measure the just-verified shared Core projection implementation against the same pinned native hypergraphz library and preserved before results. No new IR, kernel, runner, dependency, commit or push. No production source edits are included in this measurement phase. The only tracked changes are this plan bundle. CORE.YAML items touched: none; interfaces unchanged.

Reuse and provenance. Reuse Campaign, ProcessRunner, Evidence and CampaignSummary in scripts/performance_evidence/campaign.py. Verify all 399 source files against shared-incidence/source-manifest.json and the release binary SHA256 before running. Preserve earlier evidence; output is the sibling shared-incidence-measurement phase. Source snapshot and temporary Core dependency patch receipts are retained from the verified implementation, not invented commits.

Gates and matrix. Run the existing client correctness checks, then rank32 smoke before the existing complete native comparison matrix: 65,536 nodes, 1,000,000 incidences, arities 2/32/1000. Full rank includes preparation, workspace, solve and output cleanup on both sides; warm HyMeKo solve is explicitly separate. Components remain a matched control. Two warmups, nine timed batch samples per fresh process, five processes per final cell, alternating order, logical CPU 2, 180 seconds per process and 16 GiB observed process cap. Verify full vectors, residual bounds, hashes, native DLL identity and source bytes for every cell. Inspect background CPU activity before measurements; do not overlap builds.

Risks and conclusions. Report medians, IQR and worst samples, exact sizes and boundaries. Process memory includes both backends and verification, not isolated graph storage. No world-fastest or directed upstream claim. If the smoke fails, investigate before expanding. Remaining slowdown warrants a focused diagnosis, not speculative arithmetic changes or tolerance relaxation. Preserve all dirty work; rollback only this phase's plan edits using the before snapshots.

Delivery. JSON/CSV/raw logs and comparison plot, standalone PDF, source/binary identities, and verified shared ReportArchive copy. The older published Core pin remains unchanged; results describe the recorded local sources.

Exact-count preparation and seeded ordered reductions

Created-at: 2026-10-18 19:13 +0200. ETA 35–55 minutes: implement two bounded changes and targeted selection in the existing runner, numerical/allocation gates, release comparison, PDF and verified archive. User authority: continue the competitive PageRank optimization on the shared Core IR after the completed comparison.

Evidence and scope. The last exact-source campaign has full-call medians 206.23/46.07/34.63 ms versus hypergraphz 170.67/45.11/41.10 ms. Preparation still walks IR references for unit destination sums and uses a compensated outgoing-count buffer. Every gather starts compensated summation at zero. Remove this proven redundant work; do not claim a profiled percentage.

Core contract. Add an additive IncidenceAccumulator::from_first method whose default is Default + add. Ordered sum initializes from its first term and keeps all later terms and callback ordering unchanged; empty input still returns Default::total. PageRank's private compensated policy supplies an algebraically equivalent first-term state, retaining signed-zero and nonfinite behavior. No new kernel, graph, unsafe indexing, dependency or precision mode.

Unit preparation. For undirected unit weights and at most 2^53 incidences, obtain destinations from row lengths and count outgoing membership with the existing shared f64 scatter. Each integer addition is exact in this bound. Skip the redundant reference scan and compensated temporary vector. Larger counts and other policies use the existing general preparation, preserving its domain and error checks; no artificial graph-size limit. Directed roles, selected weight components, duplicates, empty arcs and isolated nodes remain intact.

Files. Core: ir/incidence/projection.rs, tests/prepared_ir_backbone.rs, CORE.YAML, SYMBOLS.md, existing plan bundle. Graph: direct_ir/pagerank.rs, existing pagerank and allocation tests, scripts/performance_evi and test_campaign.py, CORE.YAML, SYMBOLS.md, existing plan bundle. Reports extend the comparison families under exact-count-reduction; before-files preserve dirty work. Public extension is one defaulted trait method only; no existing signature changes.

Verification. Check default/seeded fold order and empty/signed-zero behavior, compensated accumulation versus the unchanged add recurrence on adversarial finite/nonfinite inputs, full independent PageRank oracles and directed/weighted cases. Check unit multiplicity, ignored payloads, projection reuse and warm zero allocations. Preserve tiny native parity metadata and full-scale rank hashes. Run affected debug/release correctness, Clippy, formatting, symbols and duplication.

Measurement. Extend existing Cell/Campaign selection with explicit subject filters; invalid, duplicate or empty selections fail, checkpoint and summary identity include the selected matrix. No second runner. Start with rank32 smoke; then five fresh processes for full and prepared rank on arities 2/32/1000, matched native control. Existing two warmups/nine samples, CPU 2, 180 s and 16 GiB guards, source/binary hashes, no concurrent build. Compare the preserved current baseline; record host load and keep non-equivalent preparation boundaries separate.

Risk and rollback. Numerical or allocation regression blocks adoption. If a candidate does not improve, diagnose or revert just that candidate to its before-file; never reset existing dirty work. The ordinary default owns any accepted speedup. Graph uses the recorded temporary local Core

patch with Cargo.lock restored. Fresh origin/github fetch succeeded; no branch change, commit or push. Deliver PDF, raw samples and source manifests, plus a byte-verified ReportArchive copy.

Two-incidence row specialization

Created-at: 2026-10-18 20:02 +0200. ETA 35–55 minutes: shared pair operations, numerical and callback-order tests, release gates, existing 45-cell comparison, PDF and archive. Authority: user explicitly requests the proposed repair for the two-vertex case, continuing the matched performance task. No validation or convergence weakening.

Evidence. Latest full-call medians are 178.02/41.33/30.45 ms for arities 2/32/1000, versus native hypergraphz 162.98/44.42/38.25 ms. The two-slot case visits 22 million rows over 44 iterations; prepared execution is 159.80 ms. These are timing observations, not an attributed CPU profile.

Shared Core owner. Add a defaulted IncidenceAccumulator::sum_pair hook taking two lazy term closures. Its default delegates to the policy's existing sum, preserving overridden sum behavior and coefficient evaluation order. Only IncidenceRow::gather selects this hook for a two-slot row. Unroll two-slot scatter through one shared local slot operation; keep additions sequential for duplicate targets and preserve panic/partial-write behavior. No new IR or persistent index.

PageRank policy. For a finite two-term sum, the rounded sum plus a positive zero is the same final value as the completed Neumaier two-term reduction. Preserve signed-zero behavior; nonfinite sums use the existing general policy. Test this property against the unchanged add recurrence over explicit extreme values and deterministic random IEEE bit patterns. Replace division by exactly 2.0 with multiplication by 0.5 after the same damping multiplication. All other denominators retain their existing arithmetic. The destination accumulators, full-vector/mass checks, personalization and convergence contract stay unchanged.

Scope and interfaces. Core: CORE.YAML, ir/incidence/projection.rs, tests/prepared_ir_backbone.rs, SYMBOLS.md, the existing four-format plan. Graph: CORE.YAML, direct_ir/pagerank.rs, tests/pagerank.rs as needed, SYMBOLS.md, the existing plan. Public change is an additive default trait method. No new crate, dependency, unsafe block, runtime version selector, or alternate PageRank. Use the existing campaign subject selector without changing its implementation.

Tests. Verify pair callback order and custom sum dispatch, empty and non-pair rows, negative coefficients/nonfinites, duplicate scatter targets and partial writes on a missing target. Pair arithmetic must match the established recurrence for finite results and nonfinite classification. Reuse the 432-model independent oracle, directed/weighted cases, unit storage and warm zero-allocation checks. Require tiny native parity and unchanged large-case result metadata. Run scoped debug/release tests, Clippy, fmt, symbols and duplication.

Measurements. Freeze source/binary identities after correctness gates. First use the existing selected rank smoke; then five fresh processes per cell for full HyMeKo/native and prepared HyMeKo, arities 2/32/1000. Existing two warmups, nine samples, CPU 2, 180 s and 16 GiB process guards. No concurrent compilation. A small-row speedup must not conceal a material large-row regression; retain all samples and compare against preserved exact-count-reduction evidence. If a candidate fails, diagnose or revert that candidate only before delivery.

Provenance and rollback. Fresh origin/github fetch succeeded; Core fceee42 is 2/6 and Graph 921d1d9 is 1/4 versus main, one checkout each. Preserve all existing dirty work with scoped before files. Temporary local Core Cargo patch only, restore Cargo.lock. No commit/push. Reports extend the existing families under pair-incidence, with PDF and byte-verified shared archive.

Current construction and diverse graph validation

Created-at: 2026-10-18 22:16 +0200. User requests fresh construction measurements and more diverse graphs. Preparation ETA 25–40 minutes; measurement window approximately 15–25 minutes after the user signals that the machine is free. The user's explicit 2026-10-18 reply requires waiting for that signal before performance runs. Correctness tests, compilation, provenance and runnable commands are prepared first.

Scope. Rebuild the existing canonical Core construction client against the current compact references. Compare one million vertices, 1000 disjoint 1000-member hyperedges, build and build/drop, five fresh processes per backend. Use the existing frozen native construction adapter, full checksums, 20 Criterion samples per operation, existing 1 s warmup/3 s measurement target and Core monitor. Prior processes took about 12 s, so ten repeats cost approximately two minutes. Do not run the retired manual builder; no production algorithm changes are planned.

Graph inputs. Keep the three recorded 65,536-node/one-million-incidence fixed-arity cases. Extend the existing Fixture strategy with: mixed widths [2,4,8,16,130] over 31,250 edges (one million incidences); a universal hub with 31 cyclic leaf slots per 32-member edge (one million); eight connected active regions using 32,768 vertices plus 32,768 isolated vertices (one million); a 65,536-node chain (65,535 pair edges); a smaller affine graph (8192 nodes, 4096 edges of size 32, 131,072 incidences); and a larger affine graph (262,144 nodes, 125,000 edges of size 32, four million incidences). All timed edges have at least two distinct members, within the upstream component algorithm's domain. Synthetic strategies are described precisely, not labeled real-world datasets.

One owner. Extend Fixture/Membership and a private row-descriptor helper; populate only through Core IrConstruction. Preserve the old affine RNG sequence and normalized fingerprints. Extend existing native Input reporting with actual arity/degree statistics using Core Distribution. No extra IR or adjacency implementation. Keep existing vector/residual and full component-partition checks.

Controller and schema. Add validated workload selection to the existing Cell/Campaign: omitted selection keeps the historical matrix; explicit named workloads include the new catalog. Record selection in checkpoint identity and summary expectations; explicit smoke selection is honored. Extend the existing ConstructionCampaign with explicit output/repository/backend parameters, retain its old script bytes in the new report phase and reuse the existing process monitor. No new sampler or process runner. Existing completed artifacts remain preserved.

Tests before runs. Small deterministic inputs prove mixed widths, distinct targets, seed reproducibility, hub degree, intended component/isolate counts and chain topology. Keep independent PageRank oracles and warm allocation gates. Exercise new native fixtures at small scale against both backends. Test invalid/duplicate workload selections, checkpoint identity and historical summaries. Build Core bench without timing; run –verify-builders correctness mode. Run applicable Clippy, formatting, Python lint/types, symbols and duplication gates.

Measurement protocol. Wait for the user signal; no timing probes in preparation. Inspect host load and avoid compiler overlap. First run production-scale smokes on the six new fixtures for full HyMeKo rank and native components (parity checks run both backends). Then run all nine

fixtures, five existing operations, five fresh processes: 225 Graph processes/2025 samples. Existing two warmups/nine samples, CPU 2, 180 s/process and 16 GiB observed memory cap. Construction adds ten processes/400 Criterion samples. Report medians, IQR, worst values, work counts, raw samples, and host limitations. No precision change or result-dependent omission.

Files. Graph CORE.YAML, SYMBOLS.md, tests/pagerank_support/mod.rs, tests/pagerank.rs, tests/performance_evidence/native.rs, scripts/performance_evidence/ campaign.py and test_campaign.py; Core's existing report run.py; both existing plan bundles. Runtime Core/Graph algorithms and Cargo dependency pins remain untouched. Temporary local Core patch for Graph verification, restored Cargo.lock bytes.

Provenance and rollback. Fresh origin/github fetch succeeded; one checkout each, dirty work preserved. New broad-validation report phases contain before-files, task-only changes and source/binary identities. Preparation may finish with measurements explicitly pending the user. Deliver PDF readiness/results, plots and JSON/CSV, and byte-verified ReportArchive copies. No commit or push.

Execution correction, 2026-10-19 02:04 +0200. The user's quiet-host signal released the campaign. The production-scale smoke exposed an adapter admission cap of two million incidences, below the already planned four-million-incidence fixture. Correct only the benchmark-owned tests/performance_evidence/native_adapter.zig bound to four million; retain the vertex/edge caps, memory budget, upstream algorithm and Rust client unchanged. Preserve the failed smoke and original source manifest/DLL. Freeze the corrected DLL and measurement source manifest separately; require all 22 small parity records, three oversized-input rejection checks and all 12 full-scale smoke cells before the 225-process comparison. All 12 smoke cells passed. Archive the exact one-line guard correction, its two explanatory comment lines, and both identities.

Virtual incidence rows and broader algorithm evidence

Created-at: 2026-10-19 02:31 +0200. Authority: user approves the proposed shared incidence improvement and broader algorithm measurements ("nezzuk"). ETA about 60–90 minutes: bounded baseline profiles, shared representation and tests, selected five-process before/after comparisons, broader algorithm clients and PDF. This is authorization to implement and measure, not a claim that all gaps are closed.

Evidence and hypothesis. The completed broad-validation campaign has nine graph shapes, exact parity, and five repeats. HyMeKo loses full and warm PageRank on hub, region and chain cases with equal iteration counts. Candidate costs are target loads, ordered compensated reductions/scatter, and vertex updates. Capture bounded samply profiles before changing the hot path. If ETW cannot start, record that concrete limitation and inspect available equivalent evidence before making any optimization claim. Never disguise a timing observation as a CPU profile.

Owner and representation. Extend Core IncidenceProjection/IncidenceRow with lossless, data-derived virtual runs only where they improve storage/work. Keep arbitrary indexed rows, ordered coefficients, duplicates, canonical IDs and immutable IR ownership. A run is derived from actual IR references, never fixture labels or seeds. No new IR, algorithm engine, dependency or global cache. Preserve the slice accessor's contract if feasible through explicit lazy expansion; hot consumers use representation-independent traversal. Avoid per-slot dynamic dispatch and large per-row overhead on pair/sparse irregular inputs. Default only an implementation with demonstrated benefit and no unexplained regressions.

Verification. Independent reconstructed targets, forward/transpose operations, nonfinite/signed-zero and compensated-order controls; empty, singleton, pair, duplicate, contiguous, strided, wrapped/irregular and large-ID cases. Reuse Core prepared-IR tests and Graph dense numerical oracles, directed/weighted policies, allocation checks and native full-vector/residual parity. Preserve existing model validation and error precedence. Core first, then Graph with temporary local Core patch and exact Cargo.lock restoration. No unrelated consumer or RL edits.

Algorithm breadth. Extend the existing typed evidence client/runner for reachability, shortest Berge path, dual reachability and selected Dijkstra/A* queries on the existing varied fixtures. All algorithms call their current owners. Use complete output/reachability/path checks against existing independent tests or mathematical invariants; no second search implementation in the measurement client. Do not invent an upstream speed ratio where BFS/direction semantics differ.

Measurement. Start with the three losing PageRank cases and a pair/32 control, then broaden only for detected risks. Five fresh processes per comparison, 2 warmups/9 calibrated samples, raw values, median/IQR/worst; keep the existing 16 GiB observed cap, CPU 2 and 180 s child timeout. Profile windows are separate from timing windows and compilation. Larger graph workloads remain bounded; avoid quadratic s-component cases without a measured/capped size. Save source/binary/PDB identities and both winning and losing results. No confirmation request is needed for the already approved Core/Graph scope; no commit/push/merge is requested.

Files and rollback. Core CORE.YAML, incidence/projection.rs and its existing test owners; Graph CORE.YAML, direct_ir/pagerank.rs only for shared-owner integration or profile-justified

equivalent arithmetic, existing performance_evidence clients and controller/tests, SYMBOLS.md and the two existing plan bundles. Snapshot every touched file before modification. Revert only this task's candidate if evidence rejects it; preserve all earlier dirty work. Existing report families get an implicit-incidence phase; deliver PDF, plots, raw data and verified Samba archive.

Representation and module refinement, 2026-10-19 03:23 +0200. The first compressed implementation reduced storage but regressed timing. Profiles identified iterator and representation-dispatch overhead. Keep separate private indexed/run row types behind one IncidenceRowOperator/IncidenceRowVisitor contract; retain public slice access and canonical IR ownership. Stream contiguous/repeated runs and short boundaries only; long irregular strides retain indexed storage. Move existing target encoding/storage to incidence/projection/targets.rs and the existing Graph compensated Sum to direct_ir/pagerank/accumulator.rs, without copying implementations. New traversal evidence is a suite of the existing Campaign and ranking client, covering reachability, dual reachability, Berge BFS and unit Dijkstra.